

BTS SIO SLAM • 2024-2026

Portfolio Professionnel

Agrégateur de Veille Technologique

Laravel 11 · Docker · Blade · RSS/Atom

Auteur	Mekaoui Reda
Formation	BTS SIO (SLAM) – 2ème année
Lycée	Charles de Foucauld, Paris
Date	Décembre 2025
GitHub	github.com/Rivalzina75
Portfolio	portfolio-1hg9.onrender.com

Sommaire

Sommaire.....	2
1. Présentation du Projet.....	3
1.1 Contexte et Objectifs.....	3
1.2 Le Sujet de Veille Technologique.....	3
1.3 Fonctionnalités Clés.....	4
1.4 Stack Technique Résumée.....	5
2. Environnement de Développement.....	5
2.1 Architecture Docker avec Laravel Sail.....	5
2.2 Stack Technique Complète.....	6
2.3 Installation Étape par Étape.....	6
2.4 Commandes Courantes.....	7
2.5 Variables d'Environnement Importantes.....	7
3. Architecture Logicielle MVC.....	8
3.1 Pattern MVC dans Laravel.....	8
3.2 Routing (routes/web.php).....	9
3.3 Les Contrôleurs Clés.....	10
3.4 Les Vues (Blade Templates).....	10
4. Sécurité & Qualité (BTS SIO).....	11
4.1 Protection CSRF (Cross-Site Request Forgery).....	11
4.2 Validation & Sanitization.....	11
4.3 Gestion des Erreurs AJAX.....	12
5. Le Moteur de Veille Technologique.....	12
5.1 Problématique.....	12
5.2 L'Algorithme "Le Filet Dérivant".....	13
5.3 Affichage Dynamique sur le Site.....	16
5.4 Points Clés de l'Implémentation.....	16
6. Interface Utilisateur.....	16
6.1 Design System & Thème.....	16
6.2 Glassmorphism Effect.....	17
6.3 Animations.....	18
6.4 Responsive Design.....	19
Conclusion.....	19
Technologies démontrées.....	19
Compétences BTS SIO validées.....	20

1. Présentation du Projet

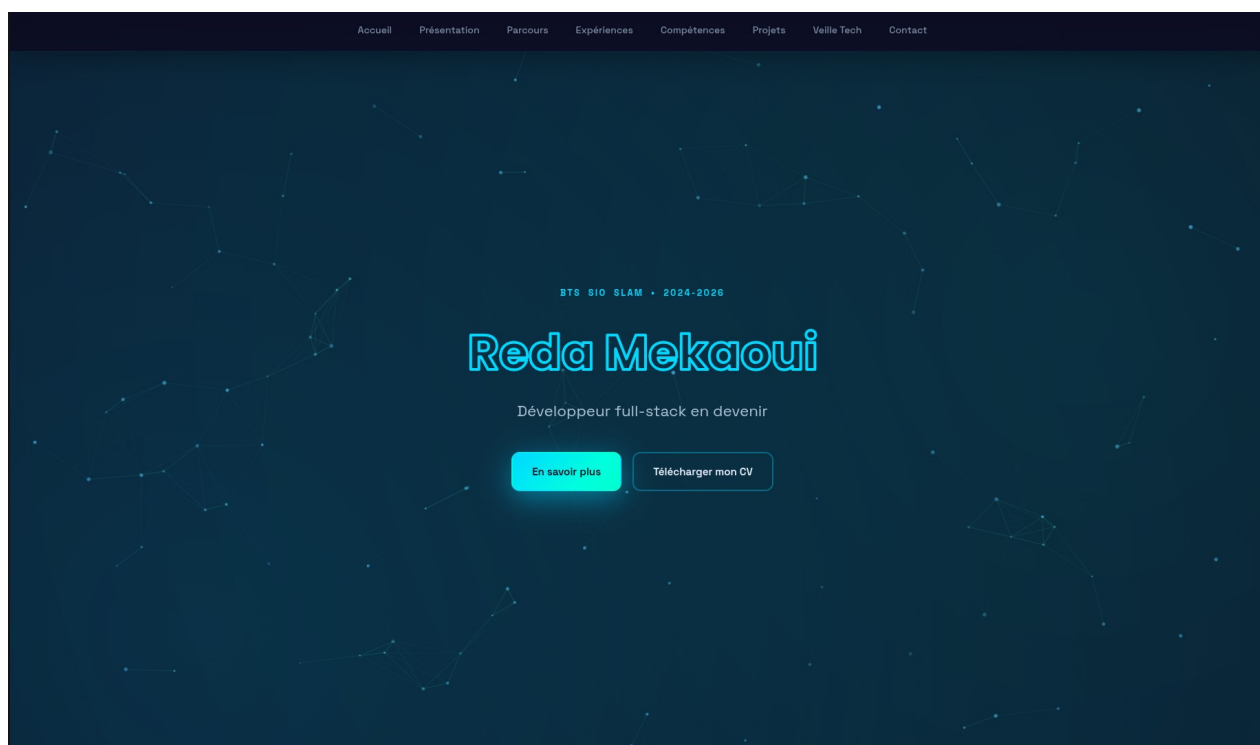
1.1 Contexte et Objectifs

Ce projet a été réalisé dans le cadre de la formation BTS SIO (Services Informatiques aux Organisations), option SLAM (Solutions Logicielles et Applications Métiers).

L'objectif dépasse la simple création d'un CV en ligne : il s'agit d'une véritable application web dynamique démontrant la maîtrise du développement Backend et Frontend avec Laravel 11.

Le site répond à trois besoins principaux :

- ▶ **Vitrine Professionnelle** : Présenter le parcours, les compétences et les projets de stage (Assuroweb, Next2You, Machina, etc.)
- ▶ **Démonstration Technique** : Prouver la capacité à utiliser un framework moderne (Laravel 11) et une architecture rigoureuse (MVC)
- ▶ **Outil de Veille Automatisé** : Intégrer un agrégateur de flux RSS intelligent pour suivre le sujet de veille en temps réel



Page d'accueil du portfolio — Hero Section avec animations de particules

1.2 Le Sujet de Veille Technologique

Une partie centrale de ce portfolio est dédiée à la veille technologique personnelle.

Thème : "L'IA générative dans les interactions UI complexes"

Problématique : Comment l'Intelligence Artificielle (comme GitHub Copilot ou V0.dev) transforme-t-elle la manière dont nous concevons et codons les interfaces utilisateurs (Frontend) ?

Solution mise en œuvre : Un algorithme capable de récupérer des articles depuis Google Alerts, de les filtrer pour ne garder que ceux parlant à la fois d'IA et d'Interface, et de les afficher dynamiquement sur le site.

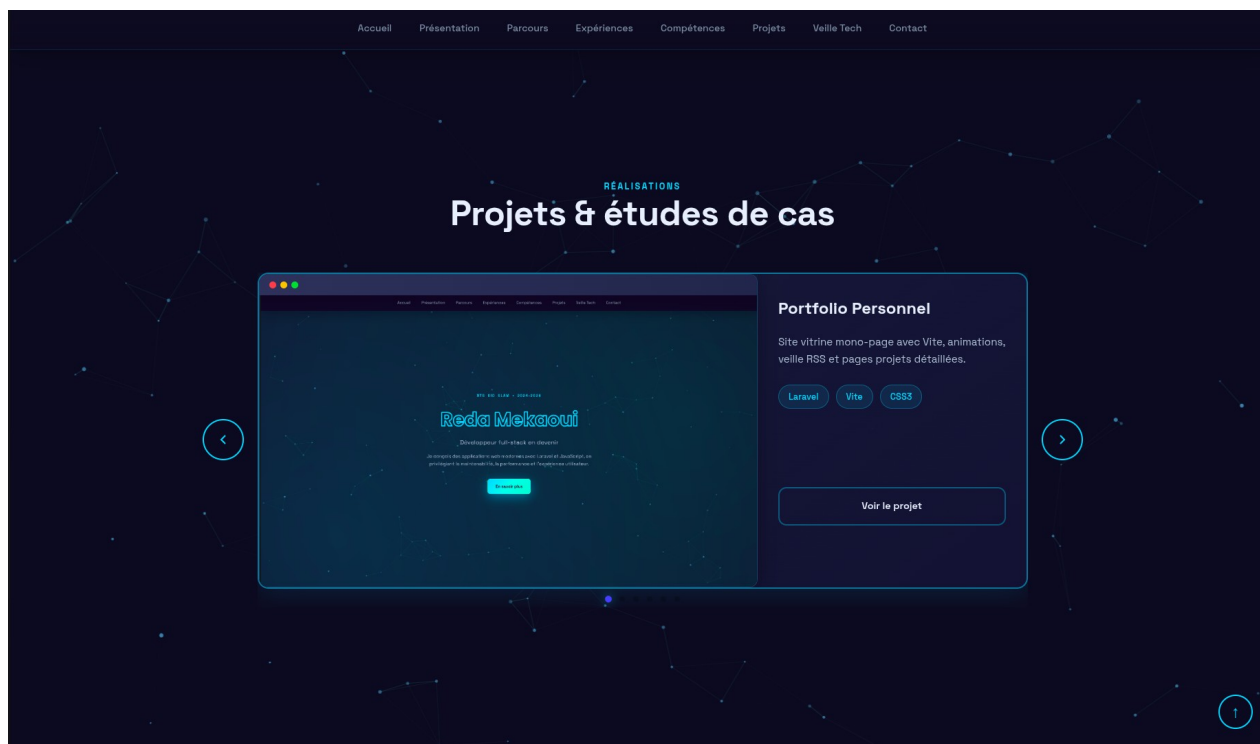


Section Veille Technologique — 6 articles filtrés automatiquement selon le sujet IA + Interactions UI

1.3 Fonctionnalités Clés

Le projet intègre plusieurs fonctionnalités avancées :

- ▶ **Carrousel de Projets** : Navigation interactive (Swiper.js) pour explorer 7 réalisations
- ▶ **Moteur de Recherche RSS** : Parsing et tri automatique de flux XML (Atom/RSS) avec filtrage intelligent
- ▶ **Formulaire de Contact AJAX** : Envoi d'emails sans rechargement de page avec validation en temps réel
- ▶ **Mode Sombre (Dark Mode)** : Design ergonomique avec thème néon cyan/violet adapté aux développeurs
- ▶ **Animations Fluides** : SVG animé (Anime.js), carrousel tactile (Swiper), particules Canvas
- ▶ **Multilingue** : Support FR/EN avec système de localisation Laravel



Carrousel interactif (Swiper.js) — projet Portfolio Personnel avec mockup, description et stack

1.4 Stack Technique Résumée

Framework	Laravel 11.x (PHP 8.2)
Frontend	Blade Templates + JavaScript Vanilla (ES6) + Vite
Base de données	MySQL 8 (Docker)
DevOps	Docker + Docker Compose + Laravel Sail
Déploiement	Render.com — portfolio-1hg9.onrender.com

2. Environnement de Développement

2.1 Architecture Docker avec Laravel Sail

Le projet repose sur une conteneurisation complète via Laravel Sail (surcouche Docker officielle de Laravel), garantissant que l'environnement de développement est identique à celui de production, sans polluer la machine hôte.

Avantages de Laravel Sail

- ✓ Aucune installation de PHP, Composer, Node.js sur la machine locale
- ✓ Isolation totale des dépendances (versions PHP, extensions)
- ✓ Configuration reproductible (commit du docker-compose.yml)

✓ Démarrage facile avec `./vendor/bin/sail up -d`

2.2 Stack Technique Complète

```
Framework:      Laravel 11.x (PHP 8.2)
Serveur Web:     Nginx (Docker)
Base de données: MySQL 8 (Docker)
Frontend:        Blade Templates + JavaScript Vanilla (ES6)
Bundler Assets:  Vite (compilation CSS/JS)
Package Manager: Composer (PHP), NPM (Node.js)
Conteneurisation: Docker + Docker Compose
```

2.3 Installation Étape par Étape

Étape 1 — Cloner le projet et installer les dépendances PHP

```
# Cloner depuis GitHub
git clone https://github.com/Rivalzina75/portfolio.git
cd portfolio

# Installer Composer via conteneur temporaire (sans PHP local)
docker run --rm \
  -u "$(id -u):$(id -g)" \
  -v "$(pwd):/var/www/html" \
  -w /var/www/html \
  laravelsail/php82-composer:latest \
  composer install --ignore-platform-reqs
```

Étape 2 — Créer le fichier d'environnement

```
# Copier le fichier exemple
cp .env.example .env

# Générer la clé d'application
./vendor/bin/sail artisan key:generate
```

Étape 3 — Lancer l'environnement Docker

```
# Démarrer tous les services (Nginx, MySQL, Redis...)
./vendor/bin/sail up -d

# Vérifier que les services tournent
./vendor/bin/sail ps

# Résultat attendu :
portfolio_laravel    Up    0.0.0.0:80->80/tcp
portfolio_mysql      Up    3306/tcp
portfolio_redis      Up    6379/tcp
```

Étape 4 — Dépendances Frontend (NPM)

```
# Installer Swiper, Anime.js, Vite...
./vendor/bin/sail npm install
```

```
# Compiler les assets en mode développement (watch)
./vendor/bin/sail npm run dev
```

Étape 5 — Migration de la base de données

```
# Créer les tables
./vendor/bin/sail artisan migrate

# Remplir avec des données de test
./vendor/bin/sail artisan db:seed
```

Étape 6 — Accéder à l'application

Ouvrir le navigateur à : <http://localhost>

2.4 Commandes Courantes

```
# Services
./vendor/bin/sail up -d      # Lancer en arrière-plan
./vendor/bin/sail down      # Arrêter tous les conteneurs

# Artisan
./vendor/bin/sail artisan list  # Voir les commandes disponibles
./vendor/bin/sail artisan tinker # Interpréteur PHP interactif

# Assets
./vendor/bin/sail npm run build  # Compiler en production
./vendor/bin/sail npm run dev    # Mode watch

# Base de données
./vendor/bin/sail mysql          # Client MySQL interactif
```

2.5 Variables d'Environnement Importantes

Fichier `.env` à configurer :

```
APP_NAME="Portfolio"
APP_ENV=local
APP_DEBUG=true
APP_URL=http://localhost

DB_CONNECTION=mysql
DB_HOST=mysql
DB_PORT=3306
DB_DATABASE=portfolio
DB_USERNAME=sail
DB_PASSWORD=password

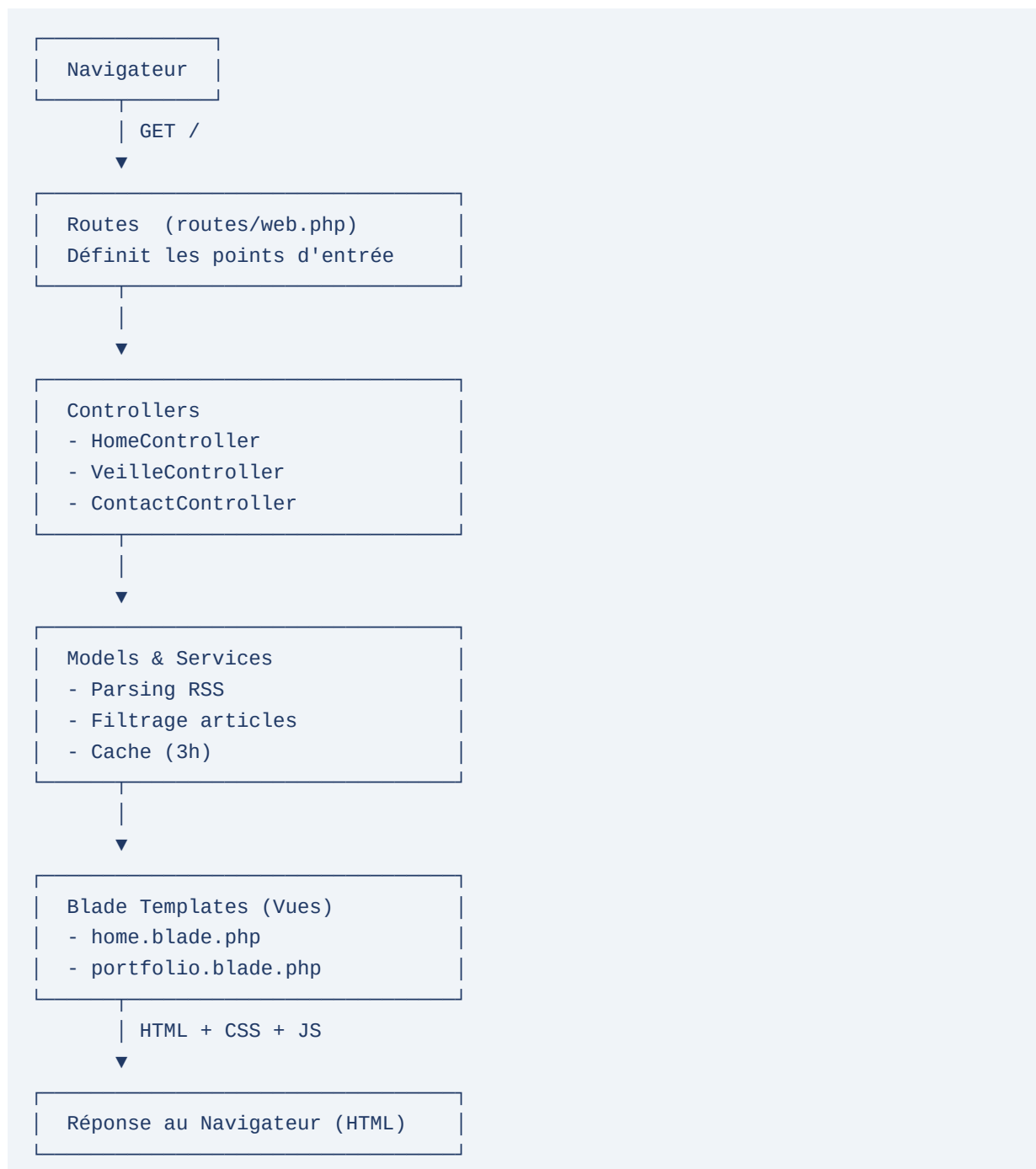
MAIL_MAILER=smtp
MAIL_HOST=smtp.gmail.com
MAIL_PORT=587
MAIL_USERNAME=votre-email@gmail.com
```

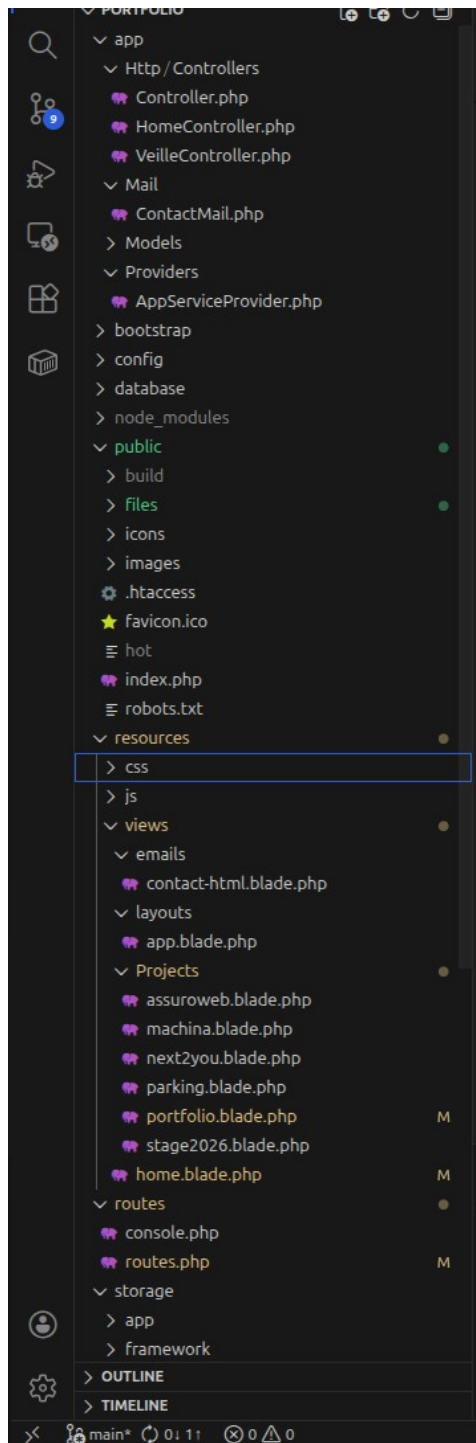
```
MAIL_PASSWORD=votre-app-password  
MAIL_FROM_ADDRESS=votre-email@gmail.com
```

3. Architecture Logicielle MVC

3.1 Pattern MVC dans Laravel

Le projet respecte strictement le pattern Modèle-Vue-Contrôleur (MVC) imposé par Laravel, favorisant la maintenance et la séparation des responsabilités.





Arborescence du projet sous VS Code — structure MVC Laravel (Controllers, Views, Routes)

3.2 Routing (routes/web.php)

Définition des points d'entrée de l'application :

```
// Page d'accueil
Route::get('/', [HomeController::class, 'index'])
    ->name('portfolio.home');

// Pages détail des projets
Route::get('/projects/{project}', function ($project) {
    return view("Projects.{ $project}");
})->name('project.{project}');
```

```
// API Veille – Récupère les articles filtrés (AJAX)
Route::get('/api/veille/articles', [VeilleController::class, 'getArticles']);

// Formulaire de contact
Route::post('/contact', [ContactController::class, 'submit'])
    ->name('portfolio.contact.submit');

// Téléchargement CV / documentation
Route::get('/files/{file}', [FileController::class, 'download'])
    ->name('portfolio.file');
```

3.3 Les Contrôleurs Clés

HomeController

```
namespace App\Http\Controllers;

class HomeController extends Controller {
    public function index() {
        // Récupère les articles depuis le cache
        $articles = Cache::get('veille_articles') ?? [];

        return view('home', ['articles' => $articles]);
    }
}
```

Responsabilités : charger la page d'accueil, injecter les données dynamiques (articles de veille), gérer la logique d'affichage.

VeilleController (Détailé en Section 5)

- ▶ Récupération des flux RSS
- ▶ Filtrage intelligent par sujet
- ▶ Gestion du cache (3h)
- ▶ Retour JSON pour AJAX

3.4 Les Vues (Blade Templates)

Utilisation du moteur de template Blade pour :

1. Héritage de layout

```
@extends('layouts.app') {{-- Héritage du layout principal --}}

@section('content')
    {{-- Contenu spécifique de la page --}}
@endsection
```

2. Boucles et conditions

```
@foreach($articles as $article)
    <article class="veille-card">
```

```

        <h3>{{ $article['title'] }}</h3>
        <p>{{ Str::limit($article['content'], 100) }}</p>
    </article>
@endforeach

```

3. Protection XSS

```

{{-- Échappe automatiquement les caractères HTML --}}
<p>{{ $user_input }}</p>    {{-- ✅ Sûr --}}

{{-- HTML brut (à utiliser avec soin) --}}
<p>{!! html_entity_decode($trusted_html) !!}</p>    {{-- ⚠️ À faire avec soin --}}

```

4. Sécurité & Qualité (BTS SIO)

4.1 Protection CSRF (Cross-Site Request Forgery)

Problème : Un attaquant pourrait créer un faux formulaire pour envoyer des emails depuis le site.

Solution Laravel : Token CSRF obligatoire sur tous les formulaires POST.

```

{{-- Chaque formulaire doit contenir ce token --}}
<form method="POST" action="{{ route('portfolio.contact.submit') }}">
    @csrf {{-- Génère <input type="hidden" name="_token" value="..."> --}}
    <input name="email" type="email" required>
</form>

```

Le serveur vérifie que le token reçu correspond à celui de la session utilisateur. Si quelqu'un soumet le formulaire depuis un autre site, la requête est rejetée (erreur 419).

4.2 Validation & Sanitization

Côté Backend — FormRequest

```

namespace App\Http\Requests;

class ContactRequest extends FormRequest {
    public function rules() {
        return [
            'name'      => 'required|string|max:100',
            'email'     => 'required|email|max:255',
            'subject'  => 'required|string|max:200',
            'message'  => 'required|string|min:10|max:5000',
        ];
    }
}

```

Règles appliquées : required (champ obligatoire), email (format RFC 5322), max:100 (limite caractères), min:10 (minimum requis). Si un champ ne respecte pas les règles → HTTP 422.

Côté Veille Technologique — Nettoyage RSS

```

// Nettoyer le contenu HTML d'un article RSS potentiellement malveillant

```

```
$content = strip_tags($article['description']);  
$content = htmlspecialchars($content, ENT_QUOTES, 'UTF-8');
```

Un flux RSS compromis pourrait contenir du JavaScript. En supprimant les tags HTML, on neutralise ce risque XSS.

4.3 Gestion des Erreurs AJAX

```
document.getElementById('contactForm')  
  .addEventListener('submit', async (e) => {  
    e.preventDefault();  
  
    const formData = new FormData(e.target);  
  
    try {  
      const response = await fetch(e.target.action, {  
        method: 'POST',  
        body: formData  
      });  
  
      if (!response.ok) {  
        // Erreur 422 = validation échouée  
        const errors = await response.json();  
        displayErrors(errors);  
      } else {  
        showSuccess('Message envoyé avec succès!');  
      }  
    } catch (error) {  
      console.error('Erreur réseau:', error);  
    }  
  });
```

Avantages : UX fluide (pas de rechargement), affichage instantané des erreurs, meilleure accessibilité.

5. Le Moteur de Veille Technologique

5.1 Problématique

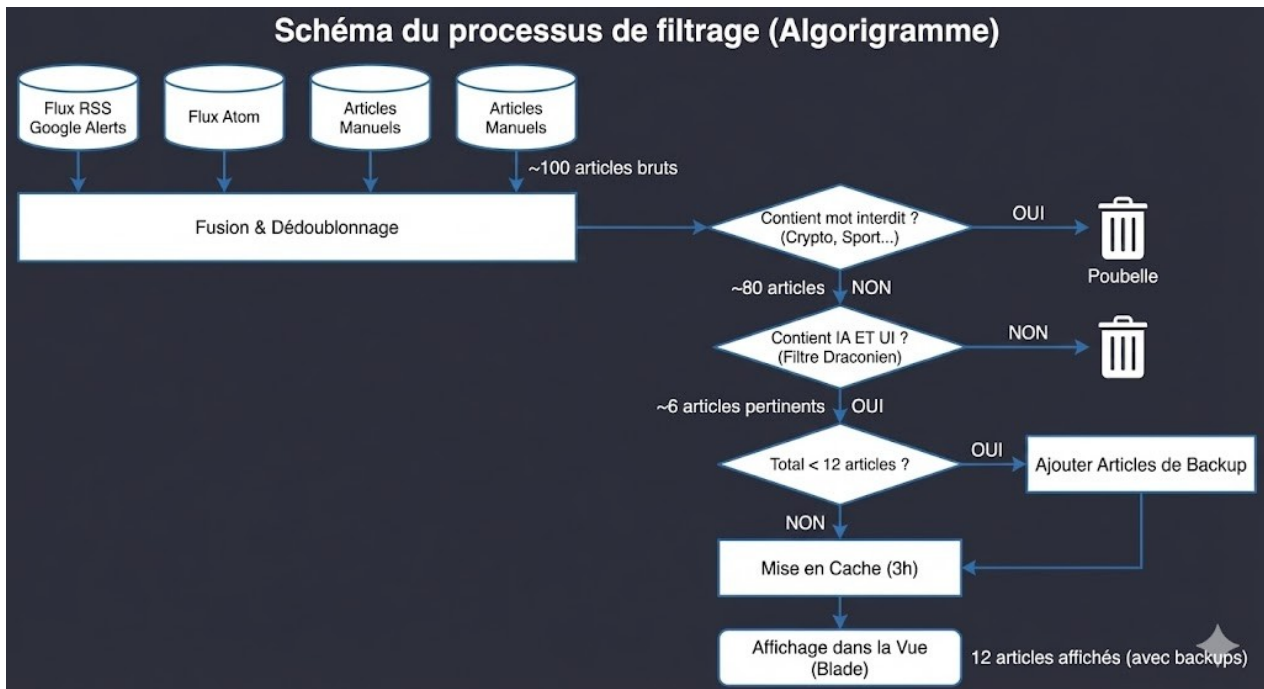
L'objectif était de suivre automatiquement le sujet de veille : "L'IA générative dans les interactions UI complexes".

Défi majeur : Les flux RSS bruts (Google Alerts) sont extrêmement "bruyants". Exemples d'articles hors-sujet récupérés :

- ▶ "Équipe IA recrute à Paris" (offre d'emploi)
- ▶ "Bitcoin atteint 50 000€" (crypto, hors-sujet)
- ▶ "Nouvelle imprimante 3D IA" (pas lié au frontend)
- ▶ "Guide : Apprendre Python pour débutants" (trop basique)

Solution : Développer un algorithme de filtrage intelligent capable de ne garder que les articles vraiment pertinents.

5.2 L'Algorithme "Le Filet Dérivant"



Algorithme du processus de filtrage — de ~100 articles bruts aux 6 articles pertinents mis en cache

Étape 1 — Agrégation Multi-Sources

Le VeilleController interroge 5 sources différentes (Google Alerts) :

```

$feeds = [
    'https://www.google.com/alerts/feeds/.../...', // "AI generate frontend code"
    'https://www.google.com/alerts/feeds/.../...', // "code quality AI assistants"
    'https://www.google.com/alerts/feeds/.../...', // "debugging AI code"
    'https://www.google.com/alerts/feeds/.../...', // "GitHub Copilot for UI"
    'https://www.google.com/alerts/feeds/.../...', // "low-code AI front-end"
];

foreach ($feeds as $feedUrl) {
    $articles = $this->parseFeed($feedUrl);
    $allArticles = array_merge($allArticles, $articles);
}
  
```

Parsing XML via SimpleXML + XPath, compatible RSS <item> et Atom <entry> :

```

private function parseFeed($feedUrl) {
    $xml = simplexml_load_string(Http::get($feedUrl)->body());
    $entries = $xml->xpath('//item | //entry');

    foreach ($entries as $entry) {
        $articles[] = [
            'title' => (string)($entry->title ?? ''),
            'link' => (string)($entry->link['href'] ?? $entry->link ?? ''),
        ];
    }
}
  
```

```

        'date'      => $this->formatDate(...),
        'content'   => strip_tags((string)($entry->description ?? $entry-
>summary ?? '')),
        'image'     => $this->extractImage($entry),
        'category' => $this->extractCategory($entry),
    ];
}
return $articles;
}

```

Étape 2 — Articles Manuels Curatés

En plus des flux RSS, 6 articles "Gold" sont sélectionnés à la main pour garantir la qualité :

```

$manualArticles = [
    [
        'title'      => 'The Future of Code: From Syntax to AI-Guided Vibe
Engineering',
        'link'       => 'https://www.startuphub.ai/...',
        'date'       => '14/12/2025',
        'category'   => 'Frontend AI',
        'is_manual' => true,
    ],
    // ... 5 autres articles importants
];
$allArticles = array_merge($allArticles, $manualArticles);

```

Étape 3 — Déduplication Intelligente

```

$uniqueArticles = [];
$seenTitles     = [];

foreach ($allArticles as $article) {
    $titleKey = strtolower(trim($article['title']));

    if (!isset($seenTitles[$titleKey])) {
        $seenTitles[$titleKey] = true;
        $uniqueArticles[]      = $article;
    }
}

```

Étape 4 — Filtrage Draconien

C'est ici que la vraie intelligence se joue. Chaque article doit satisfaire des conditions strictes :

```

$relevantArticles = array_filter($uniqueArticles, function ($article) {
    if (empty($article['image'])) return false; // Doit avoir une image

    $combined = strtolower($article['title'] . ' ' . ($article['content'] ??
''));

    // Condition 1: mot-clé IA obligatoire
    $hasAI = str_contains($combined, 'ai') || str_contains($combined, 'llm')
        || str_contains($combined, 'gpt') || str_contains($combined,

```

```
'copilot');

if (!$hasAI) return false;

// Condition 2: mot-clé Interface / UI
$compositeKw = ['ui ai', 'ai ui', 'frontend ai', 'ai frontend',
               'interaction ai', 'ai-generated'];

$hasComposite = false;
foreach ($compositeKw as $kw)
    if (str_contains($combined, $kw)) { $hasComposite = true; break; }

if ($hasComposite) return true;

// Sinon: doit avoir un contexte développeur
return str_contains($combined, 'developer') || str_contains($combined,
'code')
    || str_contains($combined, 'frontend') || str_contains($combined, 'ux');
});
```

Table de décision :

Article	IA ?	UI ?	Composite ?	Dev ?	Résultat
"GitHub Copilot for Vue"	✓	✓	✓	-	✓ GARDÉ
"AI code generator trends"	✓	✗	✗	✓	✓ GARDÉ
"Learn Python with AI"	✓	✗	✗	✗	✗ REJETÉ
"Bitcoin AI prediction"	✓	✗	✗	✗	✗ REJETÉ
"Design systems & AI"	✓	✓	✓	-	✓ GARDÉ

Étape 5 — Tri par Pertinence + Date

```
usort($relevantArticles, function ($a, $b) {
    $kw = ['ui ai', 'frontend ai', 'interaction ai', 'ai-generated'];

    $ca = strtolower($a['title'] . ' ' . ($a['content'] ?? ''));
    $cb = strtolower($b['title'] . ' ' . ($b['content'] ?? ''));

    $aHas = false; $bHas = false;
    foreach ($kw as $k) {
        if (str_contains($ca, $k)) $aHas = true;
        if (str_contains($cb, $k)) $bHas = true;
    }

    if ($aHas && !$bHas) return -1;
    if (!$aHas && $bHas) return 1;

    // À pertinence égale : plus récent en premier
    return strtotime($b['rawDate']) - strtotime($a['rawDate']);
});
```

Étape 6 — Sélection finale + Mise en cache

```
// Garder les 6 meilleurs articles
$finalArticles = array_slice($relevantArticles, 0, 6);

// Stocker en cache 3 heures (10 800 secondes)
Cache::put('veille_articles', $finalArticles, 10800);

return $finalArticles;
```

Pourquoi 3 heures ? Pas trop court (évite le bannissement IP par Google Alerts), pas trop long (articles restent frais). Optimal pour un usage personnel.

5.3 Affichage Dynamique sur le Site

```
{{-- Dans home.blade.php --}}
<div class="veille-grid">
    @foreach($articles as $article)
        <article class="veille-card">
            <div class="veille-image">
                <span style="color: var(--primary); font-weight: bold;">
                    {{ $article['category'] }}
                </span>
            </div>
            <p class="veille-title">{{ Str::limit($article['title'], 60) }}</p>
            <p class="veille-date">Publié le : {{ $article['date'] }}</p>
            <a href="{{ $article['link'] }}" target="_blank">Lire l'article</a>
        </article>
    @endforeach
</div>
```

5.4 Points Clés de l'Implémentation

- ✓ Robustesse : gestion des erreurs XML, fallback sur images par défaut
- ✓ Performance : cache 3h pour éviter les appels réseau inutiles
- ✓ Relevance : tri composite keywords + date décroissante
- ✓ Maintenabilité : code lisible, commenté, logique claire
- ✓ Sécurité : strip_tags, htmlspecialchars, validation des URLs

6. Interface Utilisateur

6.1 Design System & Thème

Le portfolio utilise un design system cohérent inspiré du dark mode néon, avec des variables CSS centralisées :

```
:root {
    --primary:      #00d9ff; /* Cyan néon */
    --accent:       #00ffcc; /* Turquoise */
    --bg-dark:      #0f1419; /* Fond très sombre */
```



```
--text:          #ffffff; /* Texte blanc */
--text-secondary: #cbd5e1;
--muted:         #94a3b8;

}
```

Typographie :

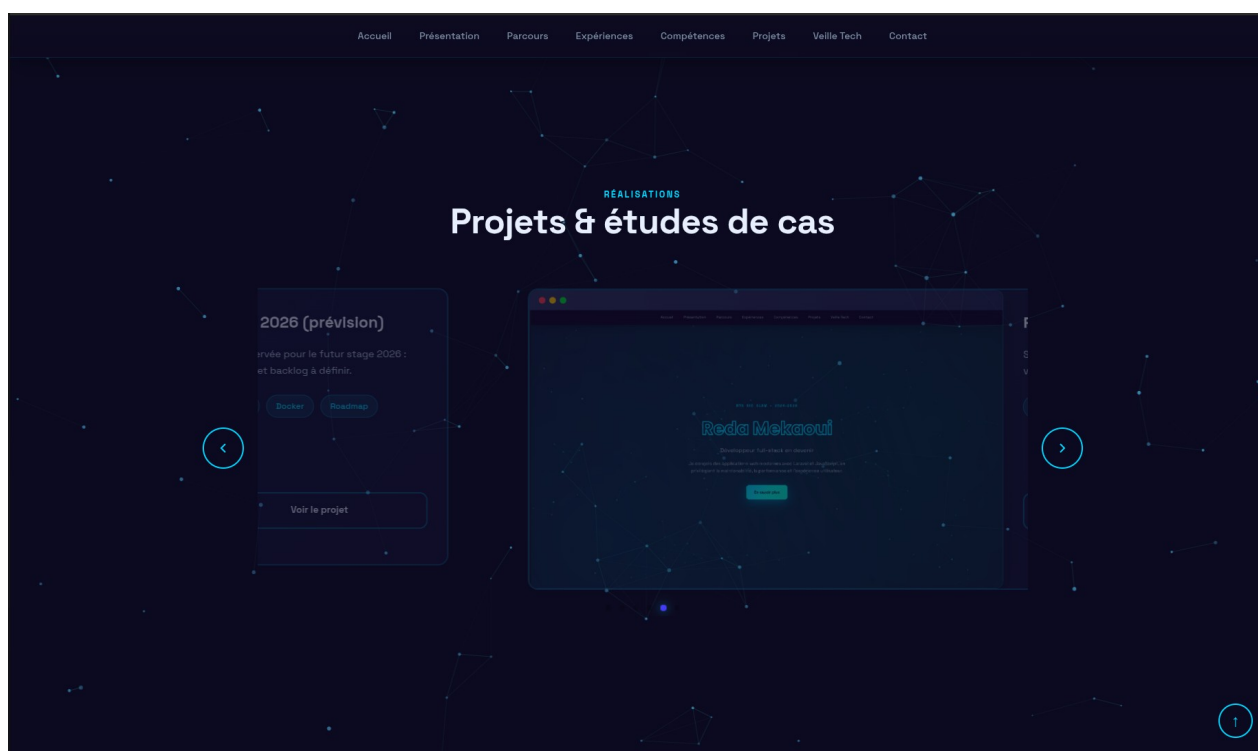
- ▶ Titres : Poppins Bold 700
- ▶ Corps : Inter Regular 400, 14-16px
- ▶ Code : JetBrains Mono (monospace)

6.2 Glassmorphism Effect

Les cartes projets utilisent l'effet Glassmorphism pour un rendu moderne et sophistiqué :

```
.project-card {
  background:      rgba(20, 25, 40, 0.6); /* Fond semi-transparent */
  backdrop-filter: blur(12px);             /* Flou arrière-plan */
  border:          1px solid rgba(0, 217, 255, 0.1);
  border-radius:   20px;
}

.project-card:hover {
  border-color: var(--primary);
  box-shadow:    0 20px 40px -10px rgba(0, 217, 255, 0.15);
  transform:     translateY(-10px);
}
```



Section Projets — carrousel avec effet Glassmorphism et animations au survol

6.3 Animations

Swiper.js — Carrousel Tactile

```
import Swiper from 'swiper';
import { Navigation, Pagination } from 'swiper/modules';

new Swiper('.projects-swiper', {
  modules: [Navigation, Pagination],
  slidesPerView: 1,
  spaceBetween: 30,
  loop: true,
  grabCursor: true,
  navigation: { nextEl: '.carousel-next', prevEl: '.carousel-prev' },
  pagination: { el: '.swiper-pagination', clickable: true },
});
```

Anime.js — Animation SVG Fluide

```
import anime from 'animejs';

anime({
  targets: '.hero-title .line',
  strokeDashoffset: [length, 0],
  duration: 2500,
  easing: 'easeInOutQuad',
  loop: true,
  direction: 'alternate'
});
```

Canvas API — Particules en Arrière-Plan

```
class Particle {
  constructor() {
    this.x = Math.random() * canvas.width;
    this.y = Math.random() * canvas.height;
    this.speedX = (Math.random() - 0.5) * 0.6;
    this.speedY = (Math.random() - 0.5) * 0.6;
  }
  draw() {
    ctx.fillStyle = 'rgba(99, 209, 255, 0.6)';
    ctx.arc(this.x, this.y, this.size, 0, Math.PI * 2);
    ctx.fill();
  }
}

const animate = () => {
  particles.forEach(p => { p.update(); p.draw(); });
  connect(); // Lignes entre particules proches
  requestAnimationFrame(animate);
};
```

6.4 Responsive Design

Le design est mobile-first, avec des breakpoints pour tablette et desktop :

```
/* Mobile first – 1 colonne */
.veille-grid {
  display:          grid;
  grid-template-columns: 1fr;
  gap:              20px;
}

/* Tablette */
@media (min-width: 768px) {
  .veille-grid { grid-template-columns: repeat(2, 1fr); }
}

/* Desktop */
@media (min-width: 1200px) {
  .veille-grid { grid-template-columns: repeat(3, 1fr); }
}
```

Conclusion

Ce projet démontre la capacité à mettre en œuvre un ensemble de compétences techniques et professionnelles cohérentes avec le référentiel BTS SIO SLAM.

- ✓ Architecturer une application Laravel structurée (MVC)
- ✓ Développer un algorithme de filtrage intelligent (veille technologique)
- ✓ Sécuriser une application web (CSRF, validation, sanitization)
- ✓ Conteneuriser un environnement de développement (Docker + Sail)
- ✓ Animer une interface fluide et moderne (Canvas, Anime.js, Swiper.js)
- ✓ Déployer une application en production (Render.com)

Technologies démontrées

Backend	Laravel 11, PHP 8.2, Blade Templates, Composer
Frontend	JavaScript ES6+, CSS3 (Glassmorphism), Vite
DevOps	Docker, Docker Compose, Laravel Sail
Data	RSS/Atom Parsing, XML, XPath, SimpleXML
Déploiement	Render.com — portfolio-1hg9.onrender.com

Compétences BTS SIO validées

- ▶ B1.1 — Gérer le patrimoine informatique
- ▶ B1.2 — Répondre aux incidents et aux demandes d'assistance
- ▶ B1.3 — Développer la présence en ligne
- ▶ B2.1 — Concevoir et développer une solution applicative (MVC, Laravel)
- ▶ B2.2 — Assurer la maintenance corrective ou évolutive
- ▶ B3.1 — Mettre en place et exploiter des mesures de sécurité (CSRF, validation)